



Universidad autónoma de baja California

Ingeniería en computación

Ingeniería en software

Practica 7 estructuras selectivas scripts

Alma Leticia Palacios Guerrero

Garcia Chávez Erik

Lunes 10 de septiembre del 2025

Procesamiento selectivo:

El procesamiento selectivo permite la ejecución de un conjunto de comandos de shell si la evaluación de una expresión es verdadera. El shell BASH tiene un conjunto de estructuras condicionales de control estructuras, similares a las que se encuentran en los lenguajes de programación, estas son test, if y case.

Comando *test* o `[]`

En la práctica la mayoría de los scripts hacen extensivo el uso de los corchetes o test para probar expresiones. Se ejecutará una acción si la expresión de prueba es verdadera.

Con `[]`

```
if [ expresión ]; then
    operaciones
```

```
fi
```

Con test

```
if test expresión; then
    operaciones
```

```
fi
```

Se ejecutará una acción si la expresión de prueba es verdadera. Si no se ejecutarán las operaciones de la parte falsa (else).

Con `[]`

```
if [ expresión ]; then
    operaciones
```

```
else
```

```
    operaciones
```

```
fi
```

Con test

```
if test expresión; then
    operaciones
```

```
else
```

```
    operaciones
```

```
fi
```

nota: el espacio entre la condición y los cachetes al inicio y al final de esta es importantes ya que sin estos al ejecutar el programa dará un error.

La siguiente estructura del if permite anidar estructuras if, para permitir la selección de múltiples alternativas. Utiliza el comando elif, el cual prueba la variable de nuevo si el primer if no fue verdadero. Si ninguna de las pruebas es verdadera, entonces se ejecuta la parte falsa del elif.

```
if [ expresión ]; then
```

```
    operaciones
```

```
elseif [ expresion ]; then
```

```
    operaciones
```

```
else
```

```
    operaciones
```

```
fi
```

compracion de cadenas

= verdadero si las cadenas son iguales

!= Verdadero si las cadenas son diferentes.

-n cadena Verdadero si la cadena no es null. //NO FUNCIONA A MENOS QUE ESTE ENTRE COMILLAS "\$1"

-z cadena Verdadero si la cadena es null (una cadena vacía)

Comparación de números

expresión1 -eq expresión2 Verdadero si las expresiones son iguales.

expresión1 -ne expresión2 Verdadero si las expresiones no son iguales.

expresión1 -gt expresión2 Verdadero si la expresión1 es mayor que expresión2.

expresión1 -ge expresión2 Verdadero si la expresión1 es mayor o igual que expresión2.

expresión1 -lt expresión2 Verdadero si la expresión1 es menor que expresión2.

expresión1 -le expresión2 Verdadero si la expresión1 es menor o igual que expresión2.

!expresión El símbolo ! niega la expresión y regresa verdadero si la expresión es falsa.

Operadores Lógicos

! Not

-a And

-o Or

Operadores de archivos

-d archivo Verdadero si el archivo es un directorio.

-e archivo Verdadero si el archivo existe.

-f archivo Verdadero si el archivo normal.

-r archivo Verdadero si el archivo se puede leer.

-s archivo Verdadero si el archivo tiene longitud mayor de 0 bytes.

-w archivo Verdadero si el archivo se puede escribir.

-x archivo Verdadero si el archivo se puede ejecutar

Los operadores lógicos && y ||.

orden && orden

Iniciando de izquierda a derecha, cada comando se ejecutará siempre y cuando el comando anterior se haya ejecutado con éxito. Si todas las ordenes se ejecutan con éxito, el enunciado AND, considerándose como un todo regresará un valor verdadero. En el momento que uno de los comandos no se ejecute correctamente, entonces se las demás ya no serán ejecutadas y la orden AND tendrá un valor falso

orden || orden

Los comandos se ejecutan comenzando de izquierda a derecha, se van ejecutando uno por uno, hasta que uno regresa verdadero. Entonces la ejecución de la lista de comandos se detiene y la orden OR regresará un valor verdadero. Si ninguno de los comandos de la lista OR regresa un valor verdadero, entonces la orden OR regresará un valor falso.

Case

La estructura **case** permite seleccionar entre alternativas mediante la comparación de una variable con varios patrones posibles. asociados a un conjunto de instrucciones que se ejecutarán cuando se encuentre una coincidencia. La estructura inicia con la palabra reservada **case**, luego una variable y la palabra **in**. Enseguida se escriben una lista de patrones, al final de cada patrón se escribe un paréntesis derecho. Después del paréntesis se listan los comandos asociados al patrón, para marcar el fin de este bloque de comandos se escribe dos veces el punto y coma. Finalmente, **case** se cierra con la palabra reservada **esac**

El caso default es cuando ninguno de las posibles opciones surja como verdadera entonces el caso que se ejecutara si no se encontró una coincidencia, El caso default se indica agregando a la lista de patrones de comparación, el asterisco como si fuera otro caso más. La sintaxis es la siguiente

case variable **in**

patrón1) instrucciones

;;

patrón2) instrucciones

;;

patrónN) instrucciones

;;

*) instrucciones

;;

esac

Dentro de un patrón se puede incluir cualquier caracter especial. Los caracteres especiales del shell son el asterisco, los corchetes, el signo de interrogación y el pipe. En el case, puede aplicar el uso de operadores lógicos para asociar más de un patrón un bloque de instrucciones.

Sintaxis

case variable **in**

patron1 | patron2) instrucciones

;;

patron3) instrucciones

;;

*) instrucciones

esac

Ciclo for-in

La estructura for-in está diseñada para hacer referencia de forma secuencial a una lista de valores. Requiere de dos operandos, estos son: una variable y una lista de valores. Cada elemento de la lista se asigna uno a uno a la variable de la estructura for-in. Cuando se llega al final de la lista el bucle se detiene, al igual que en el ciclo while, el cuerpo de ciclo inicia con la palabra reservada do y termina con done. La sintaxis del ciclo for-in es la siguiente:

for variable **in** lista_de_valores

do

instrucciones

done

Ciclo while Este ciclo es equivalente al ciclo for. Un ciclo while empieza con la palabra reservada while y va seguido por una orden que, usualmente, efectúa una prueba. Luego se escribe la palabra do y a continuación se escriben todas las instrucciones que forman el lazo, el final se especifica mediante la palabra reservada done. Su sintaxis es:

while orden

do

instrucciones

done

while (expresión)

comandos

end

Ciclo until

Este ciclo es similar al ciclo while, pero la condición se prueba al revés. En otras palabras, el ciclo continúa hasta que la condición se hace verdadera, no mientras la condición es verdad.

until condición

do

operaciones

done

1- Escriba un archivo que contenga los nombres de 5 de sus compañeros

```
pedro
juan
dmrx
hkBoy241
joshua
```

2- Escriba un script que verifique si los nombres del archivo stalker están conectados.

```
#!/bin/bash

#debo de leer lo que hay en el documento
#perimo debemos de verificar que exista el archivo
if [ -e $1 ]; then
    #el archivo entonces si existe
    while IFS= read -r name
    do
        #en este momento "name" es el que tiene el nombre del usuario
        user_name=`w | grep "$name"`
        #parametro -z indica si la cadena esta vacia
        if [ -z "$user_name" ]; then
            echo "el usuario $name no se encuentra conectado"
        else
            echo "el usuario $name esta conectado"
        fi
    done < $1
else
    echo "el archivo no existe"
fi
```

Ejecutando:

```
$ ls
bulgaria_4 fecha_naci_5 script_saludo_3 script_stalker stalker
$ ./script_stalker stalker
el usuario pedro no se encuentra conectado
el usuario juan no se encuentra conectado
el usuario dmrX esta conectado
el usuario hkBoy241 no se encuentra conectado
el usuario joshua no se encuentra conectado
$ w
 21:51:24 up 11 min,  1 user,  load average: 0.00, 0.03, 0.02
USER      TTY      FROM            LOGIN@   IDLE   JCPU   PCPU   WHAT
dmrx      pts/1    -                21:39    11:43   0.30s   0.30s  -bash
$ |
```

Explicación:

Para el script, antes de poder procesar algo debemos de verificar que el archivo en efecto exista lo cual es mediante el operador de archivo **-e** que marcara verdadero o falso si el archivo en el primero parámetro proporcionado existe dentro del directorio actual, si no existe marca que no existe, ahora si

existe lo que se debería de hacer es leer línea por línea en archivo “**stalker**” lo cual se hace con la instrucción **while IFS= read -r name**

Que significa esta parte del código, significa que como no sabemos de cuantas líneas es el archivo por lo que usamos el ciclo while y este leera hasta el final del archivo, utilizamos **IFS=** este es el **separador de campo interno** por defecto bash divide las líneas leídas en **campos** usando espacios/tabs como separador al asignar **IFS=** como vacíos desactivamos esta división lo que garantiza que cada línea se lea completo incluso son las líneas en blanco

-r es el modo de lectura, evitando que caracteres de escape sean interpretados, si hay un `\n`, `\t` por ejemplo este lo mantiene lo leer como si fuera texto plano. Y guardamos cada línea leída dentro de la variable **name**

Ahora tengo en **\$name** el nombre de usuario de x línea al cual quiero saber si esta conectado en este instante, por lo que hago una sustitución de comandos la cual se llama **user_name**, con el comando **w** se lista todos los usuarios conectados en el servidor en el momento que se consulta, tomo esa salida con la ayuda del **pipeline** filtro esa salida y verifico que dentro de los activados en este momento y el nombre que tome de x línea del archivo existe el usuario conectado en este momento, si la variable **\$user_name** esta vacía , lo cual es que no encontró al usuario conectado indicara que no esta conectado en otro caso quiere decir que tiene algo, por lo que si esta conectado.

- 3- Escriba un script que reciba de la línea de comando el nombre, apellido paterno y apellido materno e imprima el mensaje “Hola como estas + ”, si el usuario no da los tres parámetros mandar un mensaje de nombre incompleto.

```
#!/bin/bash
|
if [ -z $1 ] || [ -z $2 ]; then
    #si algunos de los 3 falta entonces entraria aqui
    echo "nombre incompleto"
else
    echo "hola como estas $*"
fi
```

Salidas:

Sin nada:

```
$ ./script_saludo_3  
nombre incompleto
```

Solo nombre:

```
$ ./script_saludo_3 erik  
nombre incompleto
```

Nombre ya apellido paterno

```
$ ./script_saludo_3 erik garcia  
hola como estas erik garcia
```

Nombre completo:

```
$ ./script_saludo_3 erik garcia chavez  
hola como estas erik garcia chavez  
$ |
```

Explicación:

Cuando se inicia el programa se debieron de haber introducido por lo menos 2 parámetros que se espera que sea el primero nombre y el primer apellido, lo que se hace es mediante un bloque condicionar **if-else** primero verifica que si alguno de los 2 parámetros que se espera esta vacío, indica que el nombre no esta completo, esto con ayuda del operador **OR <||>** en otro caso que se hayan introducido 2 parámetros que se espera que es el nombre y primer apellido entonces si se introdujo el nombre aceptable por lo que pasa a saludar, como se podría introducir tanto apellido paterno, materno así como 2 nombres si es que tiene utilice un parámetro especial **<\$*>** el cual devuelve todos los parámetro introducidos, por lo que sea cual sea el formato, este los imprime completamente como se introdujeron en un inicio

```
echo "hola como estas $*"
```

- 4- Escriba un script que permita adivinar la capital de Bulgaria, el programa deberá terminar cuando el usuario escriba la capital correcta y muestre la cantidad de intentos realizados.

```
#!/bin/bash
echo "cual es la capital de bulgaria?"
read capital

cont=0

while [ $capital != "sofia" ];
do
    #ingreso algo, pero debemos de verificar que si lo haya hecho
    if [ -z $capital ]; then

        echo "debe de ingresar algo, vuelva a intentarlo"

    else
        echo "$capital no es la capital de bulgaria, intente de nuevo"
    fi

    cont=$((cont+1))

    echo "cual es la capital de bulgaria?"
    read capital

done

echo "fueron $cont intentos"
```

Salida:

```
$ ./bulgaria_4 amsterdam
cual es la capital de bulgaria?
amsterdam
amsterdam no es la capital de bulgaria, intente de nuevo
cual es la capital de bulgaria?
paris
paris no es la capital de bulgaria, intente de nuevo
cual es la capital de bulgaria?
atenas
atenas no es la capital de bulgaria, intente de nuevo
cual es la capital de bulgaria?
moscu
moscu no es la capital de bulgaria, intente de nuevo
cual es la capital de bulgaria?
sofia
fueron 4 intentos
```

Explicación:

El programa pide que se cuente las cantidades que el usuario intento adivinar la capital de Bulgaria la cual es “**sofia**”

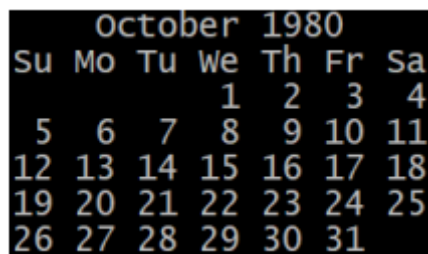
Primero se pide que introduzca la cual piensa que es la capital, utilizamos un ciclo while porque no se sabrá hasta cuando el usuario da con la capital, en cuanto el usuario pueda adivinar cual es la capital el ciclo termina y muestra cuantos intentos logro el usuario.

En caso de que no lo adivinara puede haber 2 casos, que el usuario no introdujo nada entonces dentro del ciclo primer verifica que en efecto haya introducido algo, si no introdujo cuenta como un intento, entonces la primera

condición verifica si la variable **\$capital** esta vacía entonces muestra mensaje de que debe de ingresar algo, en el otro caso solo muestra que no es la capital

Utilizamos **aritmética de bash** lo cual es necesario para realizar operaciones matemáticas en bash en este caso una suma la cual es un incremento de 1 en 1 cada vez que el usuario no ha podido adivinar la capital de Bulgaria.

- 5- Escriba un script que lea la fecha de nacimiento de una persona en formato dd/mm/aaaa, e imprima el mes del año que corresponda. Si el usuario no escribe un mes entre 1 y 12 mandar un mensaje de error. Nota: utilice el comando cal. Si el usuario escribe 16/10/1980 , su script imprimirá:



October 1980						
Su	Mo	Tu	We	Th	Fr	Sa
			1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30	31	

```
#!/bin/bash

echo "ingrese su fecha de nacimiento dd/mm/aaaa"
read date

if [ -z "$date" ]; then
    echo "debe de ingresar una fecha"
else
    if [[ $date =~ ^[0-9]{2}/[0-9]{2}/[0-9]{4}$ ]]; then
        day=`echo $date | cut -d '/' -f1`
        month=`echo $date | cut -d '/' -f2`
        year=`echo $date | cut -d '/' -f3`

        if [ $month -gt 12 ] || [ $month -lt 1 ]; then
            echo "mes incorrecto, debe ser entre 1 - 12"
        else
            cal $month $year
        fi
    else
        echo "formato incorrecto: formate dd/mm/aaa"
    fi
fi
```

Salida:

```
$ ./fecha_naci_5
ingrese su fecha de nacimiento dd/mm/aaaa
23/11
formato incorrecto: formate dd/mm/aaa

ingrese su fecha de nacimiento dd/mm/aaaa
23/11/2001
    November 2001
Su Mo Tu We Th Fr Sa
      1  2  3
 4  5  6  7  8  9 10
11 12 13 14 15 16 17
18 19 20 21 22 23 24
25 26 27 28 29 30
```

Explicación:

El programa le pide al usuario su fecha de nacimiento, pero este debe cumplir un formato el cual es **dd/mm/aaaa**.

Primero le pide al usuario su fecha de nacimiento, en el cual pueden pasar 2 situaciones en donde el usuario no ingresa nada entonces el sistema indicara que debe de ingresar una fecha.

El otro caso es que ingrese algo, pero ahora debemos de verificar que la entrada sea en el formato correcto, esto lo podemos hacer mediante una **expresión regular**, este es una secuencia de caracteres que forman un patrón de búsqueda, esto para validar la entrada de datos en el programa, la cual es la siguiente condición:

```
if [[ $date =~ ^[0-9]{2}/[0-9]{2}/[0-9]{4}$ ]];
```

Lo que se está haciendo aquí es comparar una cadena con la expresión regular, para realizar esto es necesario el operador **=~**

La expresión regular que se está planteando dice que la cadena debe de iniciar y debe de terminar con el formato dato, en la primera sección se tiene **[0-9]{2}/** lo cual indica que se espera que en ese espacio deberían de ir numero de 0 a 9 y tener exactamente 2 dígitos, entonces si el usuario ingresa 1 dígito si es un día antes del 10 puede que lo marque como un error debería de indicar el cero antes como **01,02,03,etc**. Lo mismo pasa con el siguiente indicando el mes y para el año debe de tener 4 caracteres exactos entre el 0-9 el **^** y **\$** indica que debe de empezar y debe de terminar así, por lo que debe de ser ese formato con exactitud

En este caso puede haber 2 opciones, el usuario ingreso algo, pero no es el formato correcto por lo que le indica al usuario que el formato no es correcto y se le da un ejemplo de como seria el formato correcto.

En el caso que se ingreso de manera correcta lo que se hace es partir la fecha entre sus propiedades las cuales es el día, el mes y el año, utilizando el comando **echo** imprimimos la fecha, con la ayuda de un **pipeline** cortamos con el comando **< cut >** la salida indicando que la barra invertida es el delimitador, con **day** tomamos la primera columna, en **month** la segunda y en **year** la tercera.

La expresión regular no verifica si la fecha esta dentro de una fecha valida, en este caso verificamos los meses, con una condicionar le decimos al programa si el mes es mayor a 12 o si es mes es menor a 1 entonces el mes esta dentro de un índice incorrecto por lo que se le hace saber al usuario que el mes debe de estar entre 1 – 12

En otro caso utilizamos el comando **cal** para imprimir el mes de ese año que es cuando nació el usuario, este comando recibe el **mes y el año**

Conclusions:

Durante el Desarrollo de la practica se pudo poner en práctica la lógica de condicionales así como de ciclos, estas es una parte muy importante en cualquier lógica de lenguaje de programación ya que es lo que le da la vida a un programa, el poder decidir el poder iterar sobre algo o para algo, porque no todos los programas son secuenciales tienen que haber variaciones porque pueden suceder una serie de circunstancias, por lo que el poner en practica el desarrollo de las estructuras condiciones y ciclos fue de gran ayuda para entender como es que el lenguaje bash las entiende y su sintaxis.

Bibliografía:

<https://www.ochobitshacenunbyte.com/2019/05/21/como-leer-un-archivo-linea-a-linea-en-bash/>